HASHING AND COLLISION RESOLUTION

BY- MS. ANJLI

ASSISTANT PROFESSOR

HASHING

- HASHING IN DATA STRUCTURE IS A TECHNIQUE USED TO EFFICIENTLY STORE AND RETRIEVE DATA IN A TABLE LIKE STRUCTURE CALLED A HASH TABLE.
- IT INVOLVES APPLYING A HASH FUNCTION TO A KEY TO COMPUTE A HASH CODE, WHICH IS THEN USED TO INDEX INTO AN ARRAY WHERE THE DATA ASSOCIATED WITH THE KEY IS STORED.
- SEARCHING TECHNIQUE WHICH CAN PROVIDE RESULT IN CONSTANT TIME.
- THIS **HASH VALUE** IS UNIQUE (OR ALMOST UNIQUE) FOR EACH INPUT AND HELPS IN:
- **FAST SEARCHING** (LIKE FINDING A CONTACT IN YOUR PHONE BY TYPING A FEW LETTERS).
- **DATA SECURITY** (PASSWORDS ARE STORED AS HASHES, NOT PLAIN TEXT).

- **HASH FUNCTION**
  IT TAKES AN INPUT (USUALLY A KEY) AND PRODUCES A FIXED SIZE STRING OF CHARACTERS, WHICH TYPICALLY REPRESENTS THE INDEX WHERE THE DATA ASSOCIATED WITH THE KEY WILL BE STORED.
- **HASH TABLE**
  A HASH-TABLE IS AN ARRAY LIKE DATA STRUCTURE WHERE EACH ELEMENT CORRESPONDS TO A POSSIBLE HASH CODE. THE SIZE OF THE HASH TABLE IS TYPICALLY CHOSEN BASED ON THE EXPECTED NUMBER OF ELEMENTS AND THE DESIRED PERFORMANCE CHARACTERISTICS.

COLLISION

- A SITUATION THAT OCCURS WHEN TWO DISTINCT KEYS HAVE THE SAME HASH VALUE(LOCATION). TO HANDLE THIS COLLISION, WE USE **COLLISION RESOLUTION TECHNIQUES**.

COLLISION RESOLUTION TECHNIQUES

- THERE ARE MAINLY TWO METHODS TO HANDLE COLLISION:

1. OPEN ADDRESSING(CLOSED HASHING)

   LINEAR PROBING

   QUADRATIC PROBING

   DOUBLE HASHING

2. CLOSED ADDRESSING(OPEN HASHING)

   CHAINING

LINEAR PROBING

- IN LINEAR PROBING, THE HASH TABLE IS SEARCHED SEQUENTIALLY THAT STARTS FROM THE ORIGINAL LOCATION OF THE HASH. IF IN CASE THE LOCATION THAT WE GET IS ALREADY OCCUPIED, THEN WE CHECK FOR THE NEXT LOCATION.
- L=H(K)
  IF COLLISION AT LOCATION L,

  LNEW=(L+I)MOD M [I=1,2,3]

- IF SLOT HASH(X) % S IS FULL, THEN WE TRY (HASH(X) + 1) % S
  IF (HASH(X) + 1) % S IS ALSO FULL, THEN WE TRY (HASH(X) + 2) % S
  IF (HASH(X) + 2) % S IS ALSO FULL, THEN WE TRY (HASH(X) + 3) % S

Linear Probing (Open Addressing)

Slot	
0	
1	
2	
3	
4	

Step 01

Empty hash table with range of hash values from **0 to 4** according to the hash function provided.

Linear Probing (Open Addressing)

Slot	
0	50
1	
2	
3	
4	

Step 02

The first key to be inserted is **50** which is mapped to **slot 0 (50%5=0)**

Linear Probing (Open Addressing)

Slot	
0	50
1	70
2	
3	
4	

Step 03

The next key is **70** which is mapped to **slot 0 (70%5=0)** but **50** is already at **slot 0** so, search for the next empty slot and insert it.

Linear Probing (Open Addressing)

Slot	
0	50
1	70
2	76
3	
4	

Step 04

The next key is **76** which is mapped to **slot 1 (76%5=1)** but **70** is already at **slot 1** so, search for the next empty slot and insert it.

Linear Probing (Open Addressing)

Slot	
0	50
1	70
2	76
3	85
4	

Step 05

The next key is **85** which is mapped to **slot 0 (85%5=0)**, but **50** is already at **slot number 0** so, search for the next empty slot and insert it. So insert it into **slot number 3**.

Linear Probing (Open Addressing)

Slot	
0	50
1	70
2	76
3	85
4	93

Step 06

The next key is **93** which is mapped to **slot 3 (93%5=3)**, but **85** is already at **slot 3** so, search for the next empty slot and insert it. So insert it into **slot number 4**.

QUADRATIC PROBING

- THIS METHOD IS ALSO KNOWN AS THE **MID-SQUARE** METHOD. IN THIS METHOD, WE LOOK FOR THE **I^2'TH** SLOT IN THE **I^{TH}** ITERATION. WE ALWAYS START FROM THE ORIGINAL HASH LOCATION. IF ONLY THE LOCATION IS OCCUPIED THEN WE CHECK THE OTHER SLOTS.
- L=H(K)

  IF THERE IS COLLISION AT L

  LNEW=(L+I2)MOD M,

  WHERE I=1,2,3,4……

- IF SLOT HASH(X) % S IS FULL, THEN WE TRY (HASH(X) + 1*1) % S
  IF (HASH(X) + 1*1) % S IS ALSO FULL, THEN WE TRY (HASH(X) + 2*2) % S
  IF (HASH(X) + 2*2) % S IS ALSO FULL, THEN WE TRY (HASH(X) + 3*3) % S

Quadratic Probing (Open Addressing)

Slot	
0	
1	
2	
3	
4	
5	
6	

Step 01

Empty hash table with range of hash values from **0 to 6** according to the hash function provided.

Quadratic Probing (Open Addressing)

Slot	
0	
1	22
2	
3	
4	
5	
6	

Step 02

The first key to be inserted is **22** which is mapped to **slot 1 (22%7=1)**

Quadratic Probing (Open Addressing)

Slot

0	
1	22
2	30
3	
4	
5	
6	

Step 03

The next key is **30** which is mapped to **slot 2 (30%7=2)**

Quadratic Probing (Open Addressing)

Slot

0		
1	22	← 1+0
2	30	← $1+1^2$
3		
4		
5	50	← $1+2^2$
6		

Step 04

The next key is **50** which is mapped to **slot 1 (50%7=1)** but slot 1 is already occupied. So, we will search **slot 1+1^2**, i.e. **1+1 = 2**. Again slot 2 is occupied, so we will search cell **1+2^2**, i.e.**1+4 = 5**,

DOUBLE HASHING

- IN THIS TECHNIQUE, THE INCREMENTS FOR THE PROBING SEQUENCE ARE COMPUTED BY USING ANOTHER HASH FUNCTION. WE USE ANOTHER HASH FUNCTION HASH2(X) AND LOOK FOR THE I*HASH2(X) SLOT IN THE $\mathbf{I^{TH}}$ ROTATION.
- IF COLLISION AT LOCATION L
  R=H2(K)
- L_{NEW}=(L+(I*R))MOD M, WHERE I=1,2,3…..

Example: Insert the keys 27, 43, 692, 72 into the Hash Table of size 7. where first hash-function is **h1(k) = k mod 7** and second hash-function is **h2(k) = 1 + (k mod 5)**

Double Hashing (Open Addressing)

Step 01

Empty hash table with range of hash values from **0 to 6** according to the hash function provided.

Double Hashing (Open Addressing)

Slot	
0	
1	
2	
3	
4	
5	
6	27

Step 02

The first key to be inserted is **27** which is mapped to **slot 6 (22%7=6).**

Double Hashing (Open Addressing)

Slot	
0	
1	43
2	
3	
4	
5	
6	27

Step 03

The next key is **43** which is mapped to **slot 1 (43%7=1).**

Double Hashing (Open Addressing)

Slot	
0	
1	43
2	692
3	
4	
5	
6	27

Step 04

The next key is **692** which is mapped to **slot 6** **(692 % 7 = 6)**, but location **6** is already occupied
Using double hashing,
hnew = [h1(692) + i * (h2(692)] % 7
= [6 + 1 * (1 + 692 % 5)] % 7
= 9 % 7
= 2
Now, as **2** is an empty slot, so we can insert 692 into **2nd slot**.

Double Hashing (Open Addressing)

Slot	
0	
1	43
2	692
3	
4	
5	72
6	27

Step 05

The next key is **72** which is mapped to **slot 2** **(72 % 7 = 2)**, but location **2** is already occupied.
Using double hashing,
hnew = [h1(72) + i * (h2(72)] % 7
= [2 + 1 * (1 + 72 % 5)] % 7
= 5 % 7
= 5,
Now, as **5** is an empty slot, so we can insert **72** into **5th slot**.

CHAINING(CLOSED ADDRESSING)

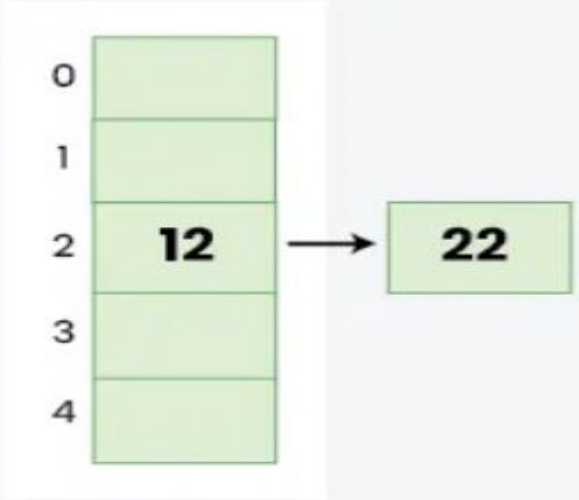
- ALL THOSE ELEMENTS THAT HASH INTO THE SAME SLOT INDEX ARE INSERTED INTO A LINKED LIST.
- NOW, WE CAN USE A KEY K TO SEARCH IN THE LINKED LIST BY JUST LINEARLY TRAVERSING. IF THE INTRINSIC KEY FOR ANY ENTRY IS EQUAL TO K THEN IT MEANS THAT WE HAVE FOUND OUR ENTRY.
- IF WE HAVE REACHED THE END OF THE LINKED LIST AND YET WE HAVEN'T FOUND OUR ENTRY THEN IT MEANS THAT THE ENTRY DOES NOT EXIST.
- HENCE, THE CONCLUSION IS THAT IN SEPARATE CHAINING, IF TWO DIFFERENT ELEMENTS HAVE THE SAME HASH VALUE THEN WE STORE BOTH THE ELEMENTS IN THE SAME LINKED LIST ONE AFTER THE OTHER.

Example: Let us consider a simple hash function as "**key mod 5**" and a sequence of keys as 12, 22, 15, 25

Separate Chaining

Step 02

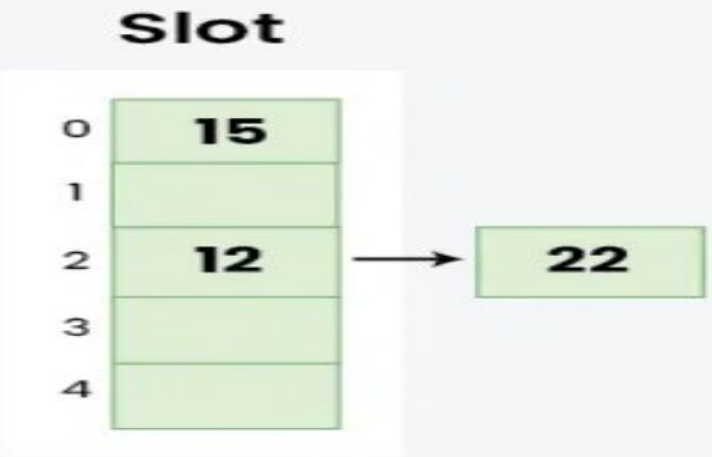
The first key to be inserted is **12** which is mapped to **slot 2 (12%5=2)**.



Step 03

The next key is **22** which is mapped to **slot 2 (22%5=2)** but **slot 2** is already occupied by **key 12**. Separate chaining will handle collision by creating a linked list to **slot 2.**

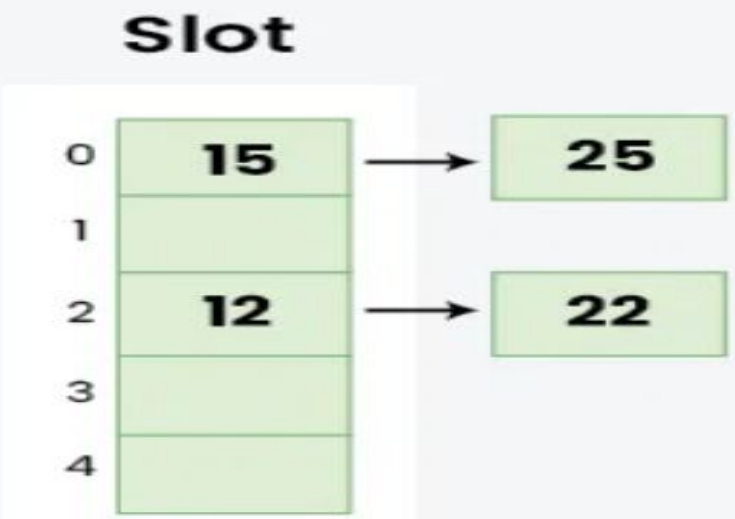
Separate Chaining



Step 04

The next key is **15** which is mapped to **slot 0 (15%5=0)**.

Separate Chaining



Step 05

The next key is **25** which is mapped to **slot 0 (25%5=0)**. But slot 0 is already occupied by **key 25**. Again, Separate chaining will handle collision by creating a linked list to **slot 2**.